



RV College of
Engineering®

Introduction to Transaction Processing Concepts and Theory

Characterizing Schedules Based on Serializability



Serial, Nonserial, and Conflict-Serializable Schedules

- A DBMS runs **many transactions** at the same time to improve **performance**.
- But concurrency can break correctness
- If multiple transactions happen but there is **no order, we cannot determine correctness**

What Is a

Schedule?

- the **actual order** in which operations (read/write) from multiple transactions execute.

Serial

Schedule

A schedule is serial if:

- **One transaction runs completely before another starts**
- **No interleaving of operations**



Properties of Serial Schedules

- Each transaction runs in isolation
- Assumes each transaction is individually correct
- Thus we say that the final database state is correct

Example : One bank teller serves one customer at a time

Problem

- No concurrency leads to poor performance
- CPU idle during I/O

Nonserial

Schedule

- Operations of transactions are interleaved

Interleaving ≠

Some interleavings are safe, some are not



Interleaved

A schedule is interleaved if:

- Operations from different transactions alternate
- Each transaction's operations are not executed consecutively

Transactions:

T₁: r(X) → w(X)

T₂: r(X) → w(X)

Serial :

Time	T ₁	T ₂
1	r(X)	
2	w(X)	
3		r(X)
4		w(X)

Interleaving

:

Time	T ₁	T ₂
1	r(X)	
2		r(Y)
3	w(X)	
4		w(Y)

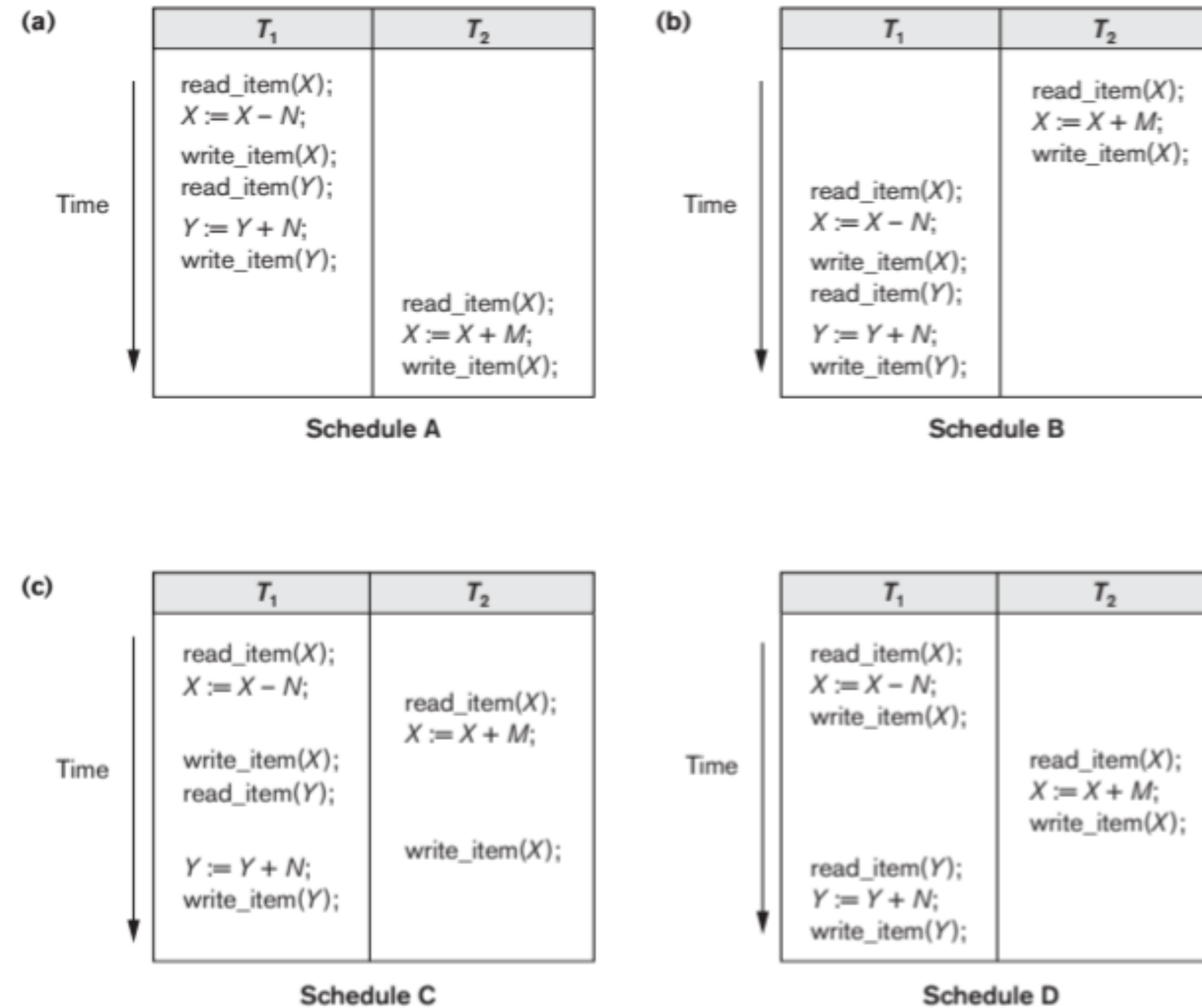
Serial, Nonserial, and Conflict-Serializable Schedules

Figure 21.5

Examples of serial and nonserial schedules involving transactions T_1 and T_2 . (a)

Serial schedule A: T_1 followed by T_2 . (b) Serial schedule B: T_2 followed by T_1 .

(c) Two nonserial schedules C and D with interleaving of operations.





Serial, Nonserial, and Conflict-Serializable Schedules

Time	T ₁ Action	T ₂ Action	DB Value of X
1	r(X)=90	—	90
2	—	r(X)=90	90
3	X = 89	—	90
4	w(X=89)	—	89
5	—	X = 92	89
6	—	w(X=92)	92
7	r(Y)=90	—	92



Explanation

- $X = 90$, $Y = 90$
- Both T_1 and T_2 read the same old value of $X = 90$
- T_1 updates X to 89, but T_2 later overwrites it with 92
- T_1 's update on X is lost
- Final state ($X = 92$, $Y = 91$) is incorrect
- This schedule is non-serializable and demonstrates the **Lost Update Problem**

Serializability

- **A schedule is serializable if it is equivalent to some serial schedule**
- It behaves like transactions ran one-by-one
- Even though they actually ran concurrently

Types of Schedule Equivalence

1. Result Equivalence

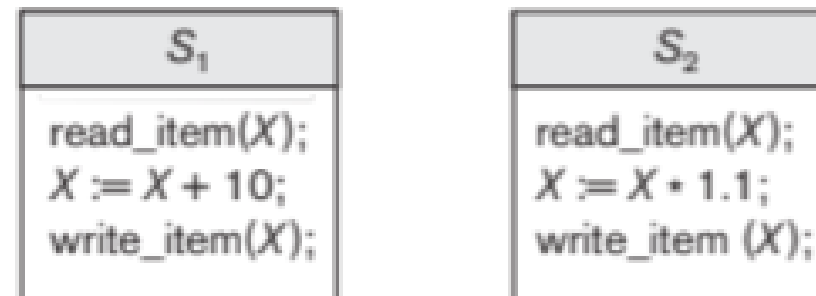
Two schedules are result-equivalent if:

- They produce the same final database

state

Figure 21.6

Two schedules that are result equivalent for the initial value of $X = 100$ but are not result equivalent in general.



- Might match results by accident

- Depends on initial values

**Not
reliable**

2. Conflict Equivalence

Two operations conflict if:

- From different transactions
- Access same data item
- At least one is a write

Examples:

- $r_1(X)$ vs $w_2(X)$
- $w_1(X)$ vs $w_2(X)$
- $r_1(X)$ vs $r_2(X)$



Serial, Nonserial, and Conflict-Serializable Schedules

Schedule A

- No interleaving
- Transactions execute one after another
- Conflicts exist but impose only one order

Serializable (already serial)

Schedule C

- $r_2(X)$ occurs before $w_1(X) \rightarrow$ conflict $\rightarrow$ forces $T_2 \rightarrow T_1$
- $w_1(X)$ occurs before $w_2(X) \rightarrow$ conflict $\rightarrow$ forces $T_1 \rightarrow T_2$
- Conflicts force opposite orders
- Reordering is not allowed

Not serializable (conflicting order)

Schedule B

- Same as Schedule A but reversed order
- Still no interleaving
- Conflicts impose only one order

Serializable (already serial)

Schedule D

- Interleaved execution
- All conflicting operations on X occur in order T_1 before T_2
- No conflict forces T_2 before T_1
- Serializable (equivalent to $T_1 \rightarrow T_2$)



**RV College of
Engineering®**

Testing for Conflict Serializability of a Schedule



Testing for Conflict Serializability of a Schedule

Need For Testing

Instead of guessing serializability, we:

- **Look only at read(X) and write(X)**
- **Capture ordering constraints caused by conflicts**
- **Represent them using a graph**
- **Check for cycles**

Precedence

Also called serializability

Graph

A directed graph $G = (N, E)$ where:

N (nodes) = transactions $\{T_1, T_2, \dots, T_n\}$

E (edges) = ordering constraints caused by conflicts

An edge $T_i \rightarrow T_j$ means:

T_i must occur before T_j in any equivalent serial schedule



Testing for Conflict Serializability of a Schedule

Algorithm

m

Step 1: Create Nodes

- For every **transaction** T_i in the schedule, create a node T_i .

Step 2: Add Edges for Conflicts(Per Transaction Data Item)

- Add an edge $T_i \rightarrow T_j$ if any of the following occurs on the same data item X :
 - a. $r_i(X)$ before $w_j(X)$
 - b. $w_i(X)$ before $r_j(X)$
 - c. $w_i(X)$ before $w_j(X)$

Step 3: Check for Cycles

- **No cycle** $\rightarrow$ schedule is conflict-serializable
- **Cycle exists** $\rightarrow$ schedule is not serializable

Testing for Conflict Serializability of a Schedule

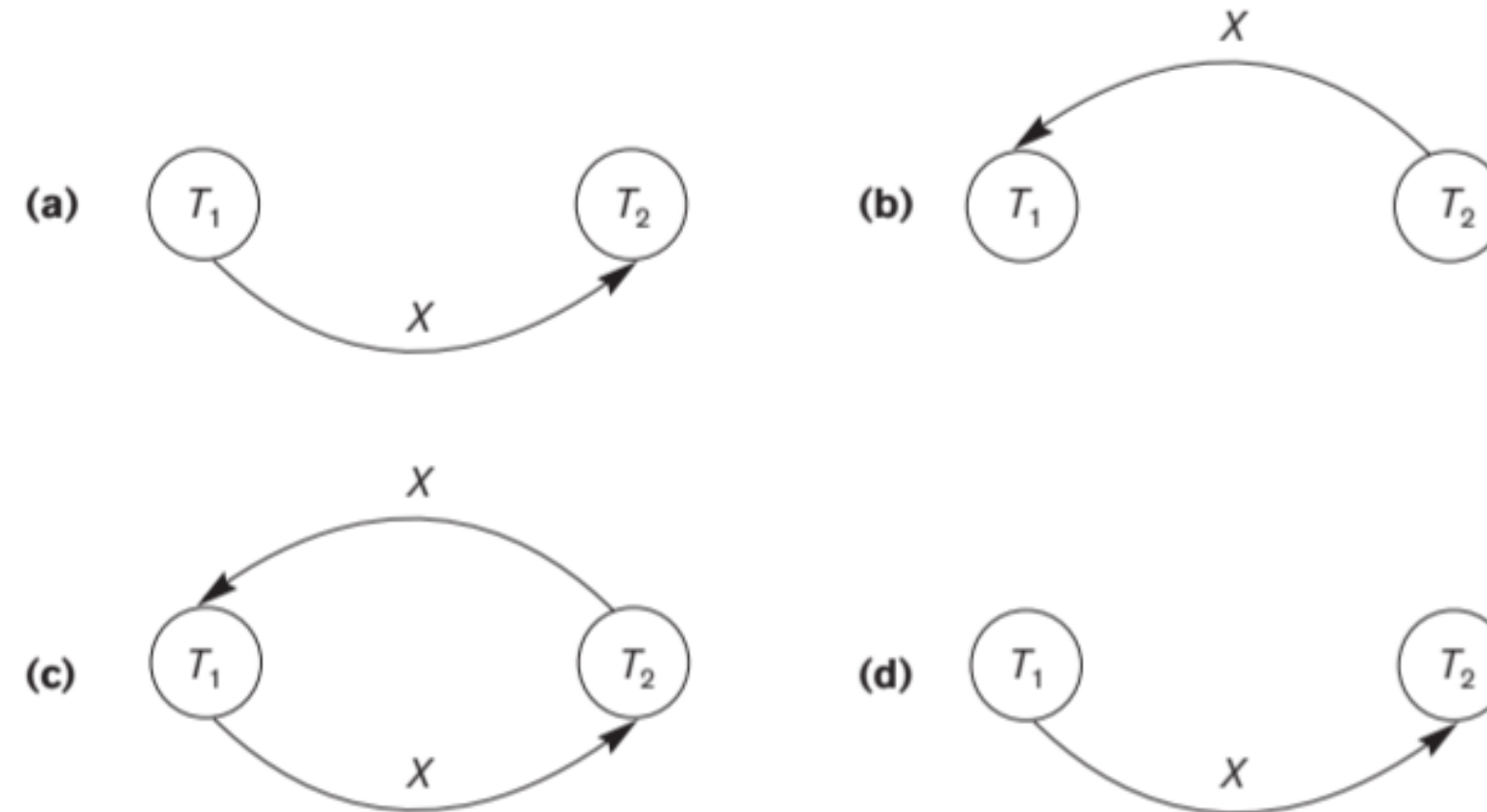


Figure 21.7

Constructing the precedence graphs for schedules A to D from Figure 21.5 to test for conflict serializability. (a) Precedence graph for serial schedule A. (b) Precedence graph for serial schedule B. (c) Precedence graph for schedule C (not serializable). (d) Precedence graph for schedule D (serializable, equivalent to schedule A).

Testing for Conflict Serializability of a Schedule

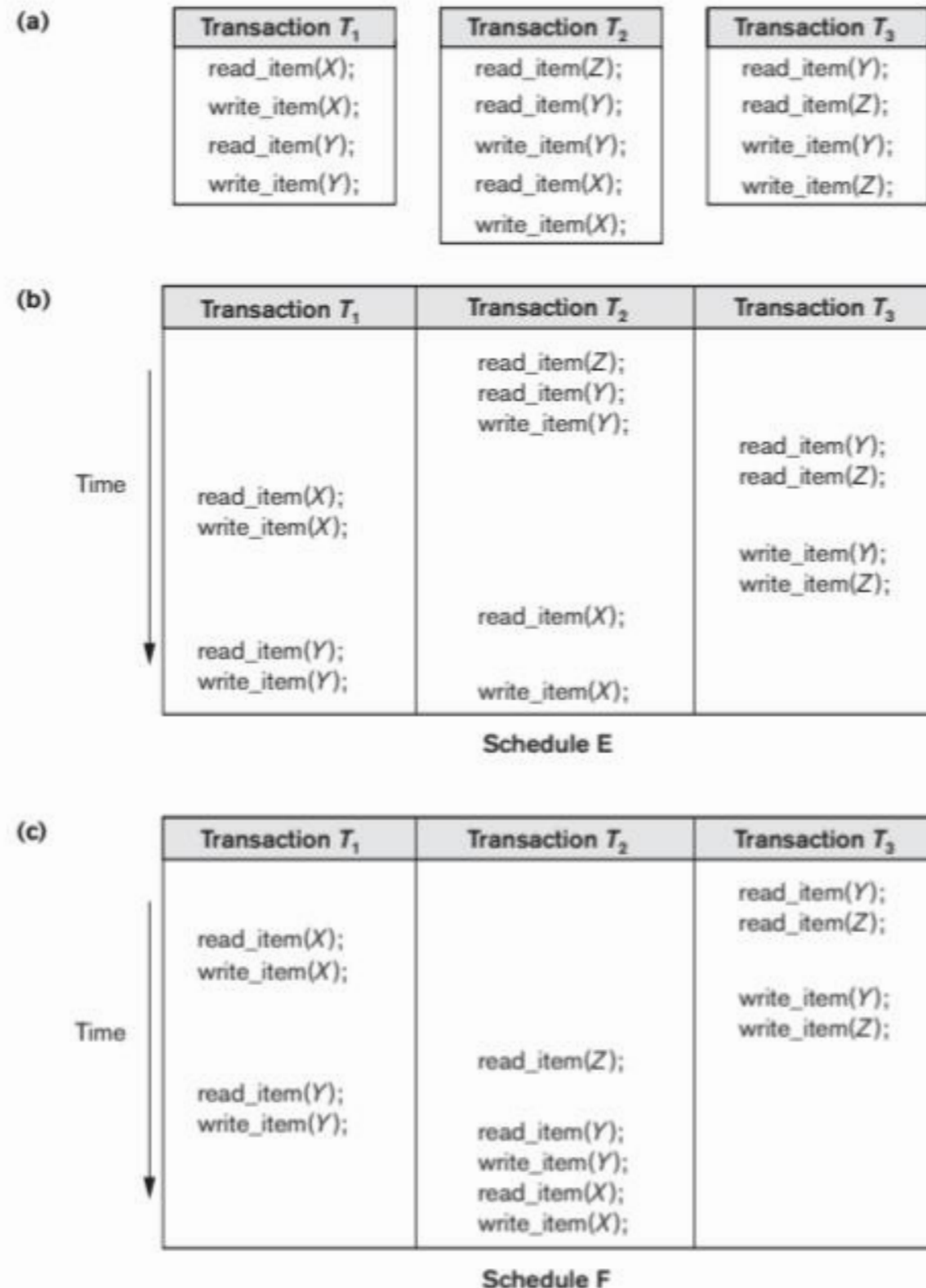
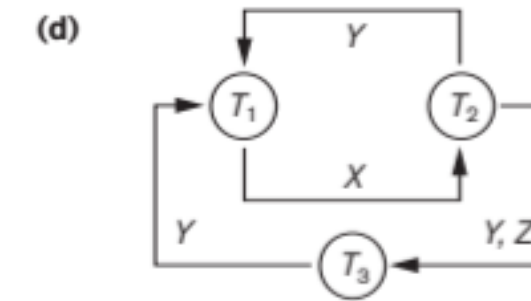


Figure 21.8
Another example of
serializability testing.
(a) The read and write
operations of three
transactions T_1 , T_2 ,
and T_3 . (b) Schedule
E. (c) Schedule F.

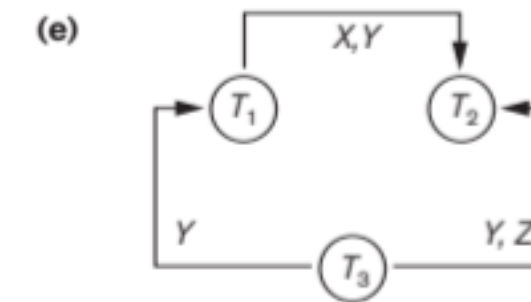


Equivalent serial schedules

None

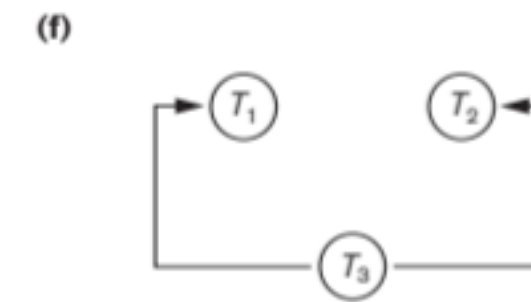
Reason

Cycle $X(T_1 \rightarrow T_2), Y(T_2 \rightarrow T_1)$
Cycle $X(T_1 \rightarrow T_2), YZ(T_2 \rightarrow T_3), Y(T_3 \rightarrow T_1)$



Equivalent serial schedules

$T_3 \rightarrow T_1 \rightarrow T_2$



Equivalent serial schedules

$T_3 \rightarrow T_1 \rightarrow T_2$

$T_3 \rightarrow T_2 \rightarrow T_1$

Figure 21.8 (continued)
Another example of serializability testing.
(d) Precedence graph for schedule E.
(e) Precedence graph for schedule F.
(f) Precedence graph with two equivalent
serial schedules.



**RV College of
Engineering®**

How Serializability Is Used for Concurrency Control



How Serializability Is Used for Concurrency Control

- Serial schedules are correct but slow
- Interleaving improves performance but risks errors
- Serializability defines correctness under concurrency
- DBMSs enforce protocols, not post-execution testing
- Following the rules guarantees serializable schedules

Testing Schedules Directly Is

Transactions are interleaved by the OS scheduler

Impractical

- Order depends on:
- System load
- CPU availability
- I/O delays
- Priorities
- Hard to even define when a schedule “starts” and



How Serializability Is Used for Concurrency Control

Concurrency Control Protocols

1. Two-Phase Locking

(2PL)
Lock data item before read/write
Growing phase → shrinking phase
Guarantees conflict serializability
Used in most commercial DBMSs

3. Multiversion Protocols

Multiple versions of data items maintained
Readers access old versions
Readers never block writers
High read concurrency

2. Timestamp Ordering

Each transaction gets a unique timestamp
Conflicts resolved using timestamp order
Older transactions get priority
No deadlocks

4. Validation (Certification)

Transactions run without locks
Conflicts checked at commit time
Commit allowed only if no conflict
Also called optimistic concurrency control



How Serializability Is Used for Concurrency Control

View Equivalence & View

Serializability

- Conflict serializability can be too restrictive
- Some correct schedules are rejected
- View serializability is more general

View Equivalence (S and S')

- Schedules are view-equivalent if all hold:
- **Same transactions, same operations**
- **Same reads-from: every read gets its value from the same write (or initial value)**
- **Same final write: last writer of each data item is the same**

Blind Write

- Write without reading the item first
- Value written does not depend on old value
- Enables view serializability beyond conflict serializability



How Serializability Is Used for Concurrency Control

View Equivalence & View

Serializability

- **More permissive than conflict serializability**—accepts schedules with **blind writes** (write without prior read)
- **Example:** $S_v: r_1(X), w_2(X), w_1(X), w_3(X)$ It is View-serializable but not Conflict-serializable
- Why view-serializable? **Same reads & final write value as serial** $T_1 \rightarrow T_2 \rightarrow T_3$
- Why not conflict-serializable? **Conflicting writes create cycle in precedence graph**
- **All conflict-serializable $\subseteq$ View-serializable** (but not vice versa)
- **Practical limitation:** Testing is NP-hard $\rightarrow$ DBMSs use conflict serializability instead

Semantic (Application-Level) Correctness

- Some non-serializable schedules are correct due to **operation semantics** (e.g., **commutativity**)
- **Example:** $T_1: X-10, Y+10 \mid T_2: Y-20, X+20 \rightarrow$ Interleaved execution preserves totals
- Why it works: Addition/subtraction commute—order doesn't affect final result
- **Challenge:** DBMSs can't infer semantics $\rightarrow$ serializability remains general standard